

A Pilot Study of Citation Gating for Retrieval Augmented Question Answering

Lakshmi Sravani Putta

Working manuscript for author review | 2 October 2026

Abstract. A citation can reproduce source text without supporting the answer to a question. This exploratory study examines that distinction using a compact local question-answering pipeline and two post-generation acceptance rules: lowercased span containment and a question-conditioned support judgment from the same language model that generated the answer. Four policies are compared on the same cached outputs for 24 SQuAD 2.0 development questions, evenly divided between answerable and unanswerable items. Each question is supplied with its original parent paragraph; retrieval is restricted to chunks within that paragraph. The ungated policy and span gate each achieve 9/24 exact matches; the model-judged gate achieves 11/24. The latter comparison has one baseline-only and three gate-only correct cases, with a two-sided exact McNemar p-value of 0.625. Always refusing achieves 12/24 because the sample is balanced, illustrating the limitations of aggregate exact match. The span and model-judged gates leave 11/12 and 9/12 unanswerable questions with nonempty answers. The evidence is a reused, inspected pilot rather than a fresh held-out evaluation. It supports a narrow implementation-level conclusion: lexical provenance checks do not by themselves establish question-sensitive support. It does not establish a reliable performance improvement, an independent verifier, or readiness for autonomous deployment.

Keywords: retrieval augmented generation; citation support; abstention; selective question answering; pilot evaluation

1 Introduction

Retrieval augmented question answering exposes evidence alongside a generated answer, making it possible to check whether an answer is grounded in supplied material. However, several distinct questions remain: did the quotation occur in the retrieved text, does it contain the proposed answer, and does it actually support that answer to this question? A passage may satisfy the first two conditions while answering a different question, reversing a premise, or failing a quantity constraint. A citation check therefore needs an explicit operational definition.

This study compares two deliberately small post-generation rules while holding generated answers fixed. One rule checks textual containment. The other asks the generator model, in a separate call, to judge question-conditioned support from the quotation. The analysis includes an always-refuse policy, separates answerable and unanswerable cases, and examines the paired changes caused by each gate. The objective is to characterize failure modes in an auditable implementation, not to introduce a new retrieval or verification method.

The experiment was motivated by technical-support knowledge bases, but uses public reading-comprehension data rather than customer tickets. Its 24 cases were inspected during development and audit. The results should be read as an exploratory case study and a basis for designing an independent evaluation. In particular, the observed two-case aggregate gain under the model-judged gate is inconclusive and does not justify a claim of general improvement.

2 Related work

Citation evaluation and verification are established research areas. Gao et al. [1] introduce ALCE, which evaluates generated answers along dimensions including correctness and citation quality. The present pilot instead uses short extractive outputs and a lexical gold-span-in-quote proxy. That proxy should not be treated as interchangeable with semantic citation evaluation.

Song et al. [2] study grounded attribution together with appropriate refusal through Trust-Score and Trust-Align. Their work makes clear that combining citations with abstention is not a new research objective. Self-

RAG [4] also precedes this pilot in learning to retrieve, generate and critique through reflection. In contrast, this implementation applies fixed post-processing rules to cached outputs, without alignment training or learned reflection behavior.

Content support must also be separated from causal faithfulness. Wallat et al. [3] distinguish a source that supports an answer from a source on which a model genuinely relied. This pilot tests textual containment and a prompted content-support judgment. It performs no source-removal or source-substitution intervention and cannot establish genuine reliance on cited evidence.

Kamath et al. [5] study selective question answering under domain shift, motivating joint attention to coverage and correctness rather than accuracy alone. The two gates examined here are fixed Boolean policies, not calibrated confidence models. Niu et al. [6] provide RAGTruth, a human-annotated hallucination corpus, illustrating the value of evaluation grounded in independent response and span annotations. The present study has no comparable human support annotations.

Zheng et al. [7] document limitations and biases in LLM-as-a-judge evaluation. This motivates validation of the support judge, but does not prove or quantify a specific bias in the Qwen model used here. Finally, SQuAD 2.0 [8] supplies adversarial unanswerable questions paired with paragraphs; it is a reading-comprehension benchmark, not a technical-support deployment test. The contribution of the current pilot is therefore an inspectable failure analysis of a particular small implementation, not a claim of first use or comparative superiority.

3 System and acceptance rules

The implementation receives a question and its gold parent paragraph. It splits the paragraph at sentence-ending punctuation followed by whitespace and groups consecutive sentences into nonoverlapping chunks of two sentences. The question and chunks are embedded with `nomic-embed-text:latest` using the prefixes `search_query:` and `search_document:`. Vectors are normalized to unit length, with a small denominator floor; chunks are ranked by dot product with stable tie handling. Up to two chunks are retained. This is oracle-paragraph retrieval, not a search over a document collection.

The local `qwen2.5-coder:3b` generator receives the selected evidence and is instructed to return a short exact answer span and a verbatim supporting sentence in JSON. If the evidence is insufficient, the prompt requests empty strings. Generation uses temperature 0, seed 42, a 180-token generation cap and a 4,096-token context setting. Invalid JSON produces empty fields. The stored model manifest identifies a 3.1B-parameter `Q4_K_M` generator and a 137M-parameter `F16` embedding model. Temperature zero and a fixed seed do not establish deterministic behavior across runtimes and hardware.

The ungated policy returns the generated answer. The always-refuse policy returns an empty answer for every case. Both gate policies either return the original generated answer or replace it with an empty string; neither edits an answer or requests a new answer.

The span gate accepts only when the answer and quotation are nonempty strings, the stripped lowercased answer is a substring of the stripped lowercased quotation, and that quotation is a substring of at least one lowercased retrieved chunk. The code uses `Python lower()`, rather than `Unicode casefold()`, and does not apply the scorer's article or punctuation normalization. This is a lexical containment predicate. It neither represents the relation between the question and the answer nor checks whether a quotation is a complete sentence.

The model-judged gate calls the same `qwen2.5-coder:3b` model with the question, proposed answer and quotation. Its instruction asks whether the quotation, read alone, states that the proposed answer answers the question; paraphrase is allowed, but inference beyond the passage is not. The call uses temperature 0, seed 42, a 40-token cap and a 2,048-token context setting. Empty fields and malformed or non-true supported values are rejected. The gate is a second judgment from the same model, not an independent natural-language-inference system.

Importantly, the two gates are alternatives in the released evaluation. The model-judged rule is not composed with the span gate and does not independently confirm that its quotation occurs in retrieved evidence. All 24 cached quotations pass a separate lowercased source-provenance check in the offline audit, so adding that provenance condition would not change this pilot's semantic-gate outcomes. A production-oriented test would need to distinguish provenance verification from semantic support and evaluate their composition. There is no learned confidence score, threshold sweep or calibrated rejection threshold in either implemented rule.

4 Pilot data and evaluation

The pilot contains 24 items from the public SQuAD 2.0 development set [8]: four answerable and four unanswerable questions from each of Computational complexity theory, Steam engine and Packet switching. Selection is deterministic. Within each article and answerability stratum, the script selects the lowest SHA-256-ordered question identifier from each eligible paragraph, then takes the four lowest hash-ordered candidates. There are 22 distinct parent paragraphs, because two paragraphs contribute one question to each answerability stratum. Topic selection and the small, balanced sample are design choices rather than a probability sample of a deployment population.

The released evaluation compares the same cached answer and quotation for every policy. It recomputes the span gate from those outputs and reads cached support judgments for the model-judged gate. The earlier span implementation used case-sensitive matching; its September 2026 audit identified a correct answer rejected solely because of capitalization. Lowercasing changed one acceptance decision without regenerating that answer. The addition of the model-judged gate and the audited analysis reused the inspected pilot. A frozen-file manifest records content hashes, but does not turn this sequence into a prospectively untouched test.

Exact match follows the repository's SQuAD-style normalization: lowercase, remove ASCII punctuation and the articles a, an and the, and collapse whitespace. An answerable item is correct if the normalized prediction matches any annotated answer. An unanswerable item is correct if the normalized prediction is empty. Token F1 is also computed against the best annotated answer. These scoring conventions are benchmark measures; they are not independent human judgments of answer utility or full semantic correctness.

Slice counts include exact matches among the 12 answerable questions, correct refusals among the 12 unanswerable questions, and refusals on answerable questions. The repository's unsupported-answer measure counts nonempty outputs on benchmark-labeled unanswerable items. It does not enumerate every possible unsupported or incorrect claim on answerable items. This paper uses that count only with its benchmark-specific definition.

The repository also reports a value named citation precision: the fraction of nonempty quotations containing a normalized gold answer span. Here it is described as gold-span-in-quote rate to avoid conflating lexical overlap with semantic support or causal citation faithfulness. An unanswerable item has no gold answer span and therefore cannot contribute to this numerator. The proxy can miss correct paraphrases and can count a span that does not justify the proposed answer in context.

The reported Wilson intervals summarize the proportion estimates under an item-level binomial model. Paired exact McNemar tests use discordant exact-match outcomes against the ungated baseline. These are exploratory calculations. The tiny selected sample, repeated paragraphs, topical clustering and prior case inspection limit statistical interpretation. A p-value above a conventional threshold does not establish equivalence or the absence of useful effects.

5 Results

Table 1 summarizes the released cached pilot [9], independently recomputed from the stored artifacts during preparation of this draft. The span gate accepts 22 answers and the model-judged gate accepts 19. Exact match is 9/24 for the ungated policy and span gate, and 11/24 for the model-judged gate. The descriptive difference between the model-judged gate and the baseline is 2/24, or 8.3 percentage points. Neither gate exceeds the always-refuse policy's 12/24 aggregate exact match.

Table 1. Descriptive results on the same 24 cached generated answers.

Metric	Always refuse	Ungated	Span gate	Model judge
Exact match	12/24	9/24	9/24	11/24
Answerable EM	0/12	9/12	8/12	8/12
Unanswerable refusals	12/12	0/12	1/12	3/12
Nonempty on unanswerable	0/12	12/12	11/12	9/12
Answerable refusals	12/12	0/12	1/12	2/12
Answers returned	0/24	24/24	22/24	19/24
EM Wilson 95% interval	[.314, .686]	[.212, .573]	[.212, .573]	[.279, .649]

For the span gate, one previously correct answer becomes a refusal and one previously incorrect unanswerable response becomes a correct refusal. The paired exact-match comparison therefore has one discordance in each direction and exact McNemar $p = 1.000$. For the model-judged gate, one baseline-correct case becomes incorrect and three baseline-incorrect cases become correct, giving $p = 0.625$. The evidence does not distinguish a reliable gain from variation in this small pilot.

The slice results explain why aggregate accuracy alone is insufficient. The ungated policy answers all 12 unanswerable questions. The span gate suppresses one of those answers while rejecting one answerable item. The model-judged gate suppresses three unanswerable answers and rejects two answerable items, one of which the baseline already answered incorrectly. Both gates retain eight answerable exact matches. Always refusing succeeds on every unanswerable item but answers no answerable item.

The gold-span-in-quote rate is 9/24 (37.5%). This is a property of the generated quotations under the stated lexical proxy, not a validated semantic citation-precision score. The cached timings give median generation-pipeline latency of 5.67 seconds, a 95th percentile of 9.27 seconds, and median additional judging latency of 1.13 seconds. They describe the recorded local run only. No controlled latency comparison, throughput study, cost reduction or production service level was measured.

6 Illustrative failure cases

Quantity mismatch survives both checks. Item 5ad56d3e5b96ef001a10ae87 asks for three complexity classes between L and P. The answer names NL and NC, and the quotation discusses those two classes. Both gates accept it despite the unmet quantity request and the item's unanswerable label. This illustrates why answer-string occurrence is insufficient and why an additional call to the same model does not guarantee a successful semantic check.

A question premise changes the interpretation of a quote. Item 5a668172f038b7001ab0bafa asks why a dedicated path is needed, while the quoted evidence says the design precludes such a need. The span gate accepts the answer because the relevant strings occur in the quotation; the model-judged gate rejects it. This is one inspected example of a question-sensitive distinction, not evidence of systematic robustness to negation.

A correct answer can have the wrong citation. For item 571153422419e3140095557e, the generated answer to who designed Salamanca matches the annotated answer, Matthew Murray, but the quotation discusses George Stephenson and different locomotives. Both gates reject it. Under answer-only exact match, this converts a correct baseline answer into an error. Under a joint answer-and-support objective, rejecting an inadequately cited answer may be appropriate. A larger evaluation should score both objectives rather than treating their disagreement as merely a nuisance.

These cases were selected after inspection to explain mechanisms. They are not an exhaustive, blinded error taxonomy and should not be used to estimate failure-mode frequencies. Unanswerable labels are used as

provided by the benchmark; the pilot includes potentially awkward question wording, so independently adjudicated support labels would be needed for stronger semantic claims.

7 Limitations and interpretation

The main limitation is evidential scope. Twenty-four exposed development items from three topics do not estimate broad technical-support performance. Public benchmark contamination is possible. Two shared paragraphs and topic-level dependence also weaken an independent-item statistical model. The audit improves transparency about implementation and arithmetic, but cannot remove selection effects or restore an untouched test set.

The oracle parent paragraph removes a central retrieval challenge. Document discovery, missing evidence, noisy corpora, conflicting sources, multi-document synthesis and realistic support workflows are outside the experiment. The label unanswerable refers to the supplied paragraph, not an assertion that a question is impossible to answer using external knowledge.

The generator and support judge share model weights and may share failure modes. A Boolean judge output is not a calibrated probability. No human support annotations or independent-verifier results are available to establish semantic gate accuracy. The model-judged gate's missing source-provenance check further prevents interpreting acceptance as full evidence validation.

Finally, the study's two quality targets can diverge: answer exact match and whether a citation adequately supports that answer. The correct-answer/wrong-citation case makes this explicit. No single aggregate score captures the cost of returning an unsupported answer, refusing a useful answer and refusing a correct but poorly cited answer. Deployment claims would require task-specific error costs and evaluation of both answer quality and evidence quality.

8 Proposed independent evaluation

The following protocol is proposed work, not completed evidence. First, freeze the implementation, prompts, preprocessing, verifier choices and primary comparison before evaluating an unseen test set. Exclude all pilot questions and parent paragraphs, and separate development and test data by document or paragraph. Add at least one domain beyond the three pilot topics and, if feasible, a permissioned support-style corpus with independently established answerability.

Second, compare the ungated and always-refuse policies with the current span rule, an independently trained entailment verifier, the same-model judge and a composed provenance-plus-support rule. Keep generated answers paired across gate conditions. Use a separate development partition for any confidence threshold; report risk and coverage across thresholds only when the verifier produces an interpretable score. A Boolean gate supplies a single operating point rather than a full risk-coverage curve.

Third, obtain blinded human annotations for answer correctness, quotation provenance and question-conditioned support, with a documented adjudication process and inter-annotator agreement. Report answerable exact match, unanswerable false acceptance, useful-answer coverage, support precision among returned answers and error categories. Include answer-only and joint answer-and-support success measures to expose metric conflicts. Preserve denominators and abstention counts for every slice.

Fourth, choose the sample size prospectively from a practically meaningful reduction in false acceptance and anticipated paired discordance, rather than treating a larger arbitrary count as sufficient. Specify the primary endpoint, multiplicity treatment and uncertainty procedure. When items share source documents, use document-aware resampling or another appropriate clustered analysis; do not apply an independent-binomial model without justification. Audit the data split and protocol freeze before revealing held-out results.

Finally, pin model digests, software versions and hardware, record full prompts and outputs, and distinguish cached rescoring from fresh embedding, generation and judging calls. Repeat fresh runs where stochastic or

runtime variation could matter. A successful larger study could support a stronger preprint; until then, this manuscript should remain an explicitly exploratory pilot.

9 Conclusion

In this cached 24-case pilot, the lexical span gate leaves aggregate exact match unchanged and the same-model support gate yields an inconclusive two-case gain. Most benchmark-unanswerable questions still receive nonempty answers under either gate. The useful observation is methodological: provenance, answer containment and question-conditioned support are separate properties, and answer-only accuracy can reward a different behavior from citation adequacy. The released artifacts make these distinctions inspectable, but an independent larger evaluation is required before claiming reliable improvement or suitability for autonomous use.

Appendix A Reproducibility and provenance

The inspected public snapshot is commit 5c0ce92dc839061a57fe859f5a1888bd8cddcd28 of <https://github.com/sravanni369/rag-citation-gate>. The source, subset, predictions, judge outputs, summary and audit are versioned in that repository. The implementation is attributed in SOURCE.md to an adaptation of Sanjay N T's RAG & Vector Databases (2026) [10], sections 8, 38, 40 and 46. That handbook is not included in the repository; its bibliographic details and redistribution permissions should be confirmed by the author before submission. This attribution does not establish independent methodological novelty.

The subset SHA-256 is 0690a51dc72bf8df219a46010ee8db37f161463f30ed027af68237f4cd3390a8. The released model manifest records generator digest f72c60cabf6237b07f6e632b2c48d533cef25eda2efbd34bed21c5e9c01e6225 and embedding digest 0a109f422b47e3a30ba2b10eca18548e944e8a23073ee3f3e947efcf3c45e59f. Full file hashes are in results/frozen_protocol.json. SQuAD-derived data is identified by the repository as CC BY-SA 4.0; preserve source attribution and the applicable license when redistributing it.

The repository documents a September 17, 2026 fresh v1 rerun whose 24 predictions matched cached predictions, plus two repeated cases in its audit. Those are repository-recorded repeatability checks, not additional independent test cases. The v2 span revision reused the original model outputs. A separate offline reproduction on 2 October 2026 used local commit b39169e4ff2411606d3f1a855d14fc55d21b2a2e. Comparison with the public snapshot confirmed byte-identical evaluation code, input subset, prediction and judge caches, frozen protocol and stored summary; differences were limited to documentation and infographic artifacts. The existing evaluator and its unit checks ran with network access blocked, and independent calculations reproduced the reported counts and paired tests. Cache identifiers, labels, verdicts and dataset/protocol hashes were checked, and cache file hashes remained unchanged. The original audit.py was not executed because it regenerates two cases. This work is offline arithmetic and artifact verification, not a fresh model run or an independent held-out evaluation.

Before author approval, confirm the preferred author name, any permitted affiliation and contact information, the handbook's attribution and license, all experimental dates and hardware, and the scope of any required AI-assistance disclosure. No affiliation, coauthors, funding source, acknowledgment or institutional email has been assumed. This is a review draft and has not been submitted.

Appendix B AI assistance and author review

Generative AI tools assisted with literature synthesis, manuscript preparation and code and results auditing during development of this working draft. This assistance includes substantive drafting rather than language correction alone. The author must verify the sources, interpretations, computations and final text before approving any submission. This draft does not assert that this verification or final author approval has been completed.

The final disclosure should identify the tools and scope accurately and be approved by the author. The intended first deposit is arXiv; its current guidance requires disclosure of significant generative AI use. The applicable guidance is <https://info.arxiv.org/help/moderation/index.html#policy-for-authors-use-of-generative-ai-language-tools>. Repository or venue eligibility is separate from the scientific limitations described in this manuscript.

References

- [1] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. EMNLP. <https://aclanthology.org/2023.emnlp-main.398/>
- [2] Maojia Song, Shang Hong Sim, Rishabh Bhardwaj, Hai Leong Chieu, Navonil Majumder, and Soujanya Poria. 2025. Measuring and Enhancing Trustworthiness of LLMs in RAG through Grounded Attributions and Learning to Refuse. ICLR. arXiv:2409.11242. <https://arxiv.org/abs/2409.11242>
- [3] Jonas Wallat, Maria Heuss, Maarten de Rijke, and Avishek Anand. 2025. Correctness is not Faithfulness in Retrieval Augmented Generation Attributions. ICTIR. DOI: 10.1145/3731120.3744592. <https://arxiv.org/abs/2412.18004>
- [4] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR. <https://arxiv.org/abs/2310.11511>
- [5] Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective Question Answering under Domain Shift. ACL. DOI: 10.18653/v1/2020.acl-main.503. <https://aclanthology.org/2020.acl-main.503/>
- [6] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL. <https://aclanthology.org/2024.acl-long.585/>
- [7] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685. <https://arxiv.org/abs/2306.05685>
- [8] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. ACL. DOI: 10.18653/v1/P18-2124. <https://aclanthology.org/P18-2124/>
- [9] Lakshmi Sravani Putta. 2026. rag-citation-gate. Public code and experimental artifacts. Inspected commit 5c0ce92dc839061a57fe859f5a1888bd8cddcd28. <https://github.com/sravanni369/rag-citation-gate>
- [10] Sanjay N T. 2026. RAG & Vector Databases: Deep study and interview edition: formulas, techniques, failure diagnosis, system design, and runnable patterns. Production Handbook. Sections 8, 38, 40 and 46. Attribution as recorded in repository SOURCE.md; author confirmation of bibliographic and permission details remains outstanding.